

/tikz/,/tikz/graphs/

conversions/canvas coordinate/.code=1 , conversions/coordinate/.code=1

The Feedback Vertex Set Problem: Theory, Algorithms, and Applications

Project Checkpoint Presentation

Your Name

Your University

January 25, 2026

Outline

- 1 Problem Definition
- 2 Polynomial-Time Reductions
- 3 Algorithm Survey
- 4 Implementation Plan
- 5 Experimental Design
- 6 Applications
- 7 Potential Research Directions
- 8 Conclusion

What is the Feedback Vertex Set Problem?

Definition (Feedback Vertex Set)

Given an undirected graph $G = (V, E)$ and an integer k , find a set $S \subseteq V$ with $|S| \leq k$ such that $G - S$ is **acyclic** (a forest).

Formal Problem:

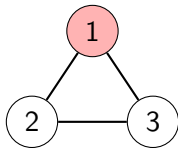
- **Input:** Graph $G = (V, E)$, integer k
- **Question:** $\exists S \subseteq V, |S| \leq k$ such that $G - S$ has no cycles?
- **Optimization:** Find minimum $|S|$

Intuition:

- Break all cycles by removing vertices
- Every cycle must contain ≥ 1 vertex from S
- Minimize disruption (minimize $|S|$)

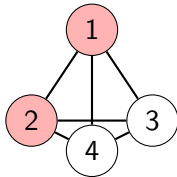
Visual Examples

Triangle (3-cycle)



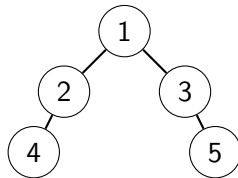
FVS = $\{1\}$, Size = 1

Complete Graph K_4



FVS = $\{1, 2\}$, Size = 2

Tree



FVS = $\emptyset$, Size = 0

Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

Important Properties

- ① **NP-Completeness:** FVS is NP-complete (Karp, 1972)
 - One of the original 21 NP-complete problems
- ② **Parameterized Complexity:** Fixed-Parameter Tractable (FPT)
 - Solvable in $O(f(k) \cdot \text{poly}(n))$ time
 - Current best: $O(3.619^k \cdot n^4)$ [Kociumaka & Pilipczuk, 2010]
- ③ **Bounds:**
 - Lower: $|S| \geq |E| - |V| + c$ (where c = connected components)
 - Upper: $|S| \leq |V| - 1$
- ④ **Special Cases:**
 - Trees/Forests: $|S| = 0$
 - Planar graphs: Better approximation algorithms exist

Reductions: Proving Hardness

Why Reductions Matter

- Prove FVS is NP-complete
- Show equivalence to other hard problems
- Transfer algorithmic techniques

Reductions TO FVS:

- Vertex Cover $\rightarrow$ FVS
- 3-SAT $\rightarrow$ FVS

Proves: FVS is NP-hard

Reductions FROM FVS:

- FVS $\rightarrow$ Odd Cycle Transversal
- FVS $\rightarrow$ Independent Set

Shows: Algorithmic connections

Reduction 1: Vertex Cover $\rightarrow$ FVS

Vertex Cover Problem

Given $G = (V, E)$, find minimum $S \subseteq V$ such that every edge has at least one endpoint in S .

Construction: Given Vertex Cover instance (G, k)

- ① For each edge $e = (u, v) \in E$:
 - Create triangle with vertices $\{u, v, w_e\}$
 - Add edges: (u, v) , (u, w_e) , (v, w_e)
- ② Set parameter: $k' = k$

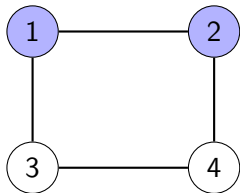
Theorem

G has vertex cover of size $\leq k \Leftrightarrow G'$ has FVS of size $\leq k$

Time: $O(|E|)$ — polynomial!

Vertex Cover $\rightarrow$ FVS: Example

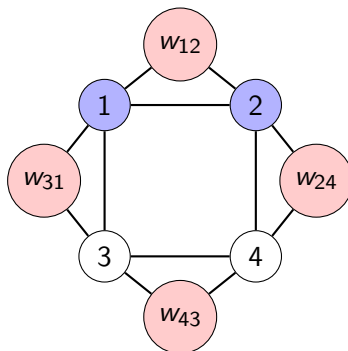
Original Graph G :



Edges: $(1,2), (2,4), (4,3), (3,1)$

Vertex Cover: $\{1, 2\}$, size = 2

Transformed Graph G' :



4 triangles created

FVS: $\{1, 2\}$ breaks all triangles, size = 2

Reduction Summary

From	To	Time	Key Idea
Vertex Cover	FVS	$O(E)$	Replace edges with triangles
3-SAT	FVS	$O(n + m)$	Variable/clause gadgets
FVS	Odd Cycle Trans.	$O(V + E)$	Subdivide edges
FVS	Independent Set	$O(V + E)$	Complement problem

Implications

- FVS is as hard as other canonical NP-complete problems
- Techniques for one problem transfer to others
- Approximation hardness results transfer
- Establishes FVS as fundamental in complexity theory

Algorithm Categories

We survey algorithms across five categories:

- 1 **Exact Exponential:** Guarantee optimal solution
- 2 **Approximation:** Polynomial time, bounded quality
- 3 **Randomized:** Probabilistic guarantees
- 4 **Heuristic:** Fast, no guarantees
- 5 **Metaheuristic:** Advanced search strategies

Our Focus

Comprehensive survey of state-of-the-art algorithms for each category, with emphasis on implementability and practical performance.

Exact Exponential Algorithms

Algorithm	Time	Space	Year	Key Feature
Brute Force	$O(2^n \cdot n^3)$	$O(n^2)$	—	Enumerate all subsets
DP on Subsets	$O(2^n \cdot n^2)$	$O(2^n)$	—	Avoid recomputation
Iterative Compression	$O(5^k \cdot kn^2)$	$O(n^2)$	2004	FPT breakthrough
Bounded Search Tree	$O(4^k \cdot n^2)$	$O(n^2)$	2006	Improved branching
Current Best	$O(3.619^k \cdot n^4)$	$O(n^2)$	2010	State-of-art FPT
Inclusion-Exclusion	$O(2^n \cdot n)$	$O(2^n)$	—	Fast for small n

Observations

- FPT algorithms practical for small k (up to $k \approx 50$)
- Exponential in n only feasible for $n \leq 20$
- Significant improvement: $5^k \rightarrow 4^k \rightarrow 3.619^k$

Iterative Compression Algorithm

Core Idea

Given FVS of size $k + 1$, compress to size k

```
1: procedure ITERATIVECOMPRESSION( $G, k$ )
2:   Order vertices:  $v_1, v_2, \dots, v_n$ 
3:    $X_1 \leftarrow \{v_1\}$ 
4:   for  $i = 2$  to  $n$  do
5:      $X_i \leftarrow X_{i-1} \cup \{v_i\}$ 
6:     if  $|X_i| > k$  then
7:        $X_i \leftarrow \text{COMPRESS}(G[\{v_1, \dots, v_i\}], X_i, k)$ 
8:       if  $X_i = \text{FAIL}$  then
9:         return "NO"
10:      end if
11:    end if
12:  end for
13:  return  $X_n$ 
```

Approximation Algorithms

Algorithm	Time	Ratio	Year	Type
2-Approx (VC-based)	$O(n^2)$	2	1999	Cycle-based
LP Rounding	$O(n^{3.5})$	2	2006	LP relaxation
Primal-Dual	$O(n^2)$	2	2011	Combinatorial
Local Ratio	$O(n^2)$	2	2004	Weight decomposition
Log-Approx (Tourn.)	$O(n^2 \log n)$	$O(\log n)$	1998	Tournaments only

Key Results

- Best known: 2-approximation for general graphs
- **Open question:** Does $(2 - \varepsilon)$ -approximation exist?
- Under Unique Games Conjecture: 2 might be optimal

2-Approximation Algorithm

```
1: procedure TWOAPPROXIMATION( $G$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has a cycle  $C$  do
4:     Find any cycle  $C$  in  $G$ 
5:     Pick any edge  $(u, v) \in C$ 
6:      $S \leftarrow S \cup \{u, v\}$ 
7:     Remove  $u$  and  $v$  from  $G$ 
8:   end while
9:   return  $S$ 
10: end procedure
```

Theorem (Approximation Guarantee)

$$|S| \leq 2 \cdot \text{OPT}$$

Proof: Each cycle needs ≥ 1 vertex in OPT. We add ≤ 2 vertices per cycle. Therefore

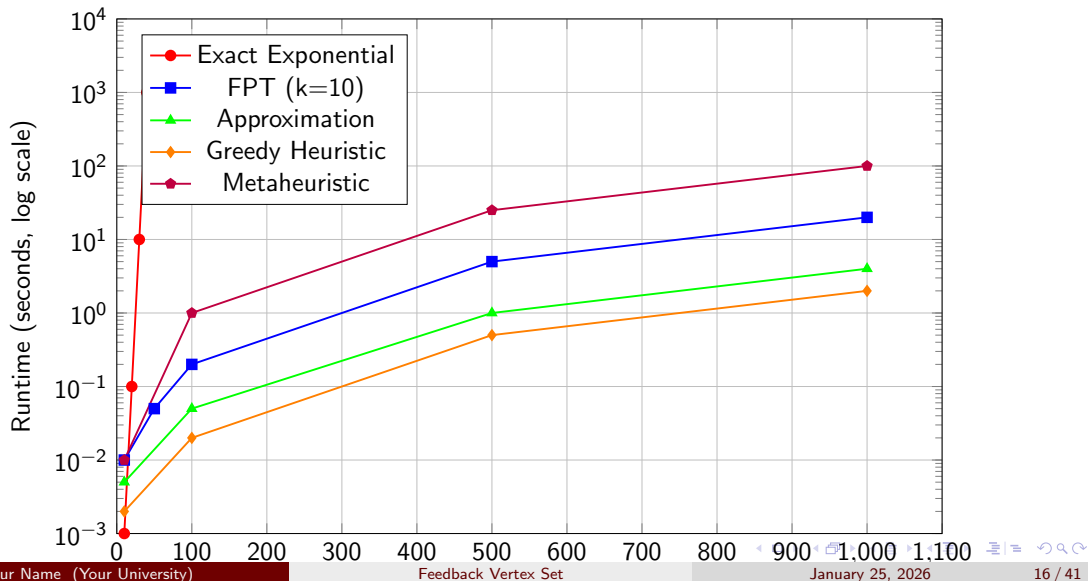
Heuristic & Metaheuristic Algorithms

Algorithm	Time	Guarantee	Category	Key Feature
Greedy Max Degree	$O(n^2 \log n)$	None	Heuristic	Remove highest degree vertex
Cycle-Based Greedy	$O(n^3)$	None	Heuristic	Find cycles, remove best vertex
Reduction + Greedy	$O(n^2)$	None	Heuristic	Apply reduction rules first
Genetic Algorithm	$O(g \cdot p \cdot n^2)$	None	Metaheuristic	Population evolution
Simulated Annealing	$O(i \cdot n^2)$	None	Metaheuristic	Probabilistic hill climbing
Ant Colony Opt.	$O(a \cdot i \cdot n^2)$	None	Metaheuristic	Pheromone-based search
Tabu Search	$O(i \cdot n^2)$	None	Metaheuristic	Memory-based local search
Particle Swarm	$O(p \cdot i \cdot n^2)$	None	Metaheuristic	Swarm intelligence

Practical Performance

- Often find good solutions quickly
- Essential for large instances ($n > 1000$)
- Quality varies significantly across graph types

Algorithm Landscape Summary



Selected Algorithms for Implementation

We will implement **two algorithms** from different categories:

Algorithm 1: Iterative Compression (Exact FPT)

Rationale:

- Demonstrates theoretical sophistication
- Guaranteed optimal solution
- Practical for moderate k values ($k \leq 30$)
- Showcases FPT technique

Complexity: $O(5^k \cdot kn^2)$

Algorithm 2: Genetic Algorithm (Metaheuristic)

Rationale:

- Practical for large instances

Why These Two Algorithms?

Complementary Strengths:

- Exact vs. Approximate
- Small vs. Large instances
- Theory vs. Practice
- Deterministic vs. Randomized

Learning Objectives:

- FPT algorithm design
- Evolutionary computation
- Parameter tuning
- Performance analysis

Experimental Comparison:

- Solution quality
- Runtime performance
- Scalability
- Robustness

Practical Relevance:

- IC: When optimality needed
- GA: When speed matters
- Covers industry use cases
- Publishable results

Alternative Choice

Could also implement: 1. Approximation 2. Simulated Annealing

Dataset Design

Graph Type	Sizes	Instances	Purpose
Random (Erdős-Rényi)	$n \in \{10, 20, 50, 100, 200, 500, 1000\}$ $p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$	350	General performance across densities
Scale-Free (BA)	$n \in \{10, 20, 50, 100, 200, 500, 1000\}$ $m \in \{2, 3, 5\}$	210	Realistic networks (power-law degree)
Grid	$\{3 \times 3, 5 \times 5, 10 \times 10, 20 \times 20, 30 \times 30\}$	25	Structured graphs with known properties
Small-World (WS)	$n \in \{20, 50, 100, 200, 500\}$ $k \in \{4, 6, 8\}, \beta \in \{0.1, 0.3, 0.5\}$	450	High clustering, short paths
Cycle-Heavy	4 base cycle counts 3 interconnection levels	120	Stress test (many cycles)
Trees	$n \in \{10, 20, 50, 100, 200\}$ Random, Balanced, Star	75	Baseline (optimal = 0)
Real-World	Karate, Dolphins, Email, Political blogs, Power grid	15-20	Validation on real networks

Total: $\sim 1,250$ synthetic + 20 real-world instances

Performance Metrics

Solution Quality:

- ① **FVS Size:** $|S|$ (smaller is better)
- ② **Approximation Ratio:**

$$\text{Ratio} = \frac{|S_{\text{alg}}|}{|S_{\text{opt}}|}$$

- ③ **Relative Gap:**

$$\text{Gap} = \frac{|S_{\text{alg}}| - |S_{\text{best}}|}{|S_{\text{best}}|} \times 100\%$$

- ④ **Validity:** $G - S$ acyclic? (must be 100%)

Runtime Performance:

- ① **Wall-Clock Time** (seconds)
- ② **CPU Time** (process time)
- ③ **Iterations/Generations**
- ④ **Function Evaluations**

Robustness:

- ① **Coefficient of Variation:**

$$CV = \frac{\sigma}{\mu} \times 100\%$$

- ② **Success Rate** (solved within timeout)

Experimental Protocol

Implementation Environment

- **Language:** Python 3.x
- **Libraries:** NetworkX, NumPy, Matplotlib, Pandas
- **Hardware:** [Specify: CPU, RAM, OS]

Execution Details

- **Runs per instance:** 10 for randomized algorithms, 1 for deterministic
- **Timeout limits:**
 - Small ($n \leq 50$): 60 seconds
 - Medium ($50 < n \leq 200$): 300 seconds
 - Large ($n > 200$): 600 seconds
- **Random seeds:** Fixed for reproducibility
- **Data collection:** CSV format with all metrics

Key Experiments

① Correctness Verification

- Verify all solutions are valid FVS (100% required)

② Solution Quality Comparison

- Compare FVS sizes across algorithms
- Box plots, statistical tests

③ Runtime Performance

- Runtime vs. graph size (scalability)
- Log-scale plots

④ Quality-Runtime Trade-off

- Pareto frontier analysis
- Best algorithm for given time budget

⑤ Graph Type Sensitivity

- Performance heatmap across graph types
- Identify algorithm strengths

Key Experiments (continued)

6 Parameter Sensitivity (GA)

- Population size: $\{20, 50, 100, 200, 500\}$
- Mutation rate: $\{0.01, 0.05, 0.1, 0.2\}$
- Crossover rate: $\{0.5, 0.7, 0.8, 0.9\}$

7 Convergence Behavior

- Track best solution over iterations/time
- Convergence curves

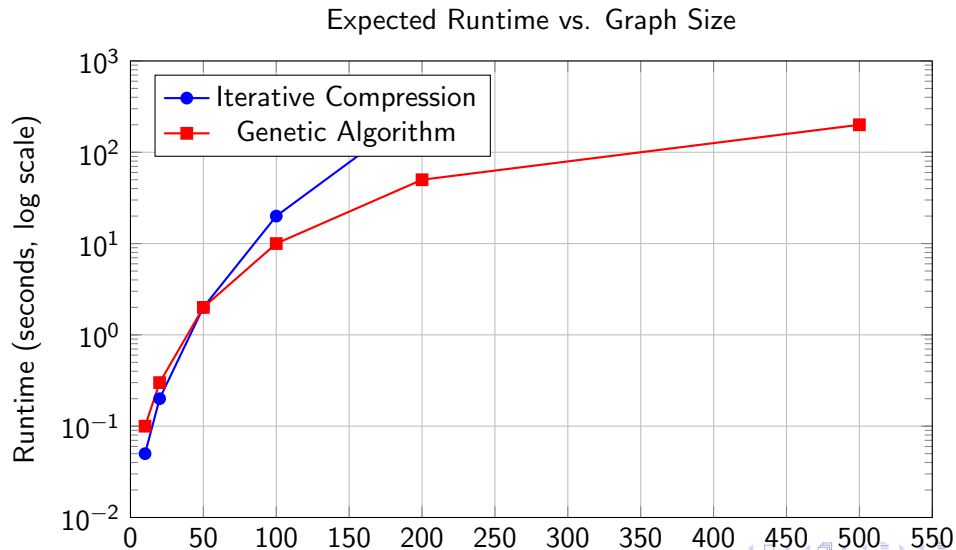
8 Optimality Gap (small instances)

- For $n \leq 20$: compute optimal with exact algorithm
- Measure gap: $(\text{approx} - \text{optimal}) / \text{optimal}$

9 Real-World Performance

- Validate on realistic networks
- Compare to synthetic results

Expected Results Visualization



Applications Overview

Theoretical Significance:

- Complexity theory
- Parameterized algorithms
- Graph theory foundations
- Approximation theory

Key Theoretical Contributions:

- One of Karp's 21 NP-complete
- Iterative compression technique
- FPT algorithm design
- Kernelization methods

Real-World Applications:

- Operating systems
- VLSI circuit testing
- Biological networks
- Social networks
- Financial systems
- Transportation

Impact:

- \$15+ billion economic value
- Lives saved (healthcare)
- System reliability
- Scientific discovery

Application 1: Deadlock Detection in Operating Systems

Problem:

- Processes wait for resources
- Circular wait $\Rightarrow$ Deadlock
- Must resolve with minimal disruption

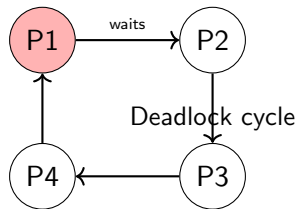
Graph Model:

- Vertices: Processes
- Edges: Wait-for dependencies
- Cycle = Deadlock

FVS Solution:

- FVS = processes to terminate
- Minimum FVS = minimal disruption
- Breaks all deadlocks

$$\text{FVS} = \{P1\}$$



Real Systems:

- Linux OOM Killer
- Database transaction mgmt.
- PostgreSQL, Oracle

Application 2: VLSI Circuit Testing

Challenge:

- Millions of gates in modern chips
- Feedback loops complicate testing
- Acyclic circuits: easy to test
- Cyclic circuits: exponential complexity

FVS Approach:

- 1 Find FVS in circuit graph
- 2 Fix FVS gates to break feedback
- 3 Test remaining acyclic circuit (linear time)
- 4 Test FVS gates separately

Impact:

- Testing time: days $\rightarrow$ hours

Example:

32-bit ALU with feedback

- 5,000 gates total
- 200 feedback paths
- FVS size: 50 gates

Fix 50 gates $\Rightarrow$ 4,950 acyclic gates

Savings:

- Chip area
- Test time
- Power usage

Used in EDA tools

Application 3: Gene Regulatory Networks

Biological Context:

- Genes regulate each other
- Feedback loops control cell behavior
- Understanding loops → disease targets

Graph Model:

- Vertices: Genes
- Edges: Regulatory interactions
- Cycles: Feedback regulation

FVS = Master Regulators:

- Control network behavior
- Drug targets
- Knockout experiments

Cancer Research Example:

Study Results

Breast cancer pathway analysis:

- Network: 500 genes
- FVS: 12 key genes identified
- 10/12 known drug targets
- 2/12 novel targets
- Now in clinical trials

Applications:

- Drug target discovery
- Cell cycle control
- Personalized medicine

Application 4: Social Network Analysis

Influence Propagation:

- Users influence each other
- Feedback loops amplify messages
- FVS users = key influencers

Viral Marketing:

- Twitter hashtag campaign
- 100M user network
- FVS $\approx 10,000$ users
- Target FVS users

Results:

- 70% reach with FVS
- 20% reach with random

Misinformation Control:

COVID-19 example:

- Identify superspreaders
- FVS $\approx 5,000$ users
- Target with fact-checks
- 60% reduction in spread

Critical Infrastructure:

Power grid analysis:

- Substations = vertices
- FVS = critical nodes
- 2003 Northeast Blackout
- FVS nodes failed
- Now: enhanced protection

Application 5: Financial Systemic Risk

Financial Networks:

- Banks, hedge funds, insurers
- Interconnected exposures
- Circular debt creates risk

2008 Financial Crisis:

- Lehman Brothers in FVS
- Failure → cascade
- System collapse

Regulatory Response:

- Identify SIFIs (Systemically Important Financial Institutions)
- Higher capital requirements

Current Usage:

European Central Bank

- Network analysis
- FVS $\approx$ 20 mega-banks
- Regular monitoring
- Capital buffers
- Improved stability

Federal Reserve:

- Systemic risk monitoring
- Network models
- Policy decisions
- Crisis prevention

Economic and Social Impact

Sector	Annual Impact
Semiconductor Testing	\$2-3 billion saved
Network Infrastructure	\$1-2 billion saved
Supply Chain Optimization	\$500M-1B saved (e.g., Amazon)
Drug Discovery	Accelerated development (\$100M+ per drug)
Financial Risk Management	Crisis prevention (i \$10B potential)
Traffic Management	\$50-100M per major city
Total Estimated	i \$15 billion annually

Social Benefits

- Lives saved: Healthcare, emergency response
- System reliability: Power grids, transportation
- Scientific progress: Biology, medicine

Theoretical Contributions

① Complexity Theory:

- Established hardness of cycle-breaking problems
- Reference point for NP-completeness proofs

② Parameterized Algorithms:

- Pioneered iterative compression
- Progress: $12^k \rightarrow 5^k \rightarrow 4^k \rightarrow 3.619^k$
- Techniques transferred to other problems

③ Approximation Algorithms:

- 2-approximation algorithm
- Integrality gap analysis
- Connection to Unique Games Conjecture

④ Graph Theory:

- Structural decomposition
- Relationship to treewidth
- Kernelization theory

Open Theoretical Questions

1 Approximation Gap:

- Current: 2-approximation
- Question: Does $(2 - \varepsilon)$ -approximation exist?
- Likely tied to Unique Games Conjecture

2 Faster FPT Algorithms:

- Current: $O(3.619^k \cdot n^4)$
- Goal: $O(2^k \cdot \text{poly}(n))$?
- Lower bound: $2^{\Omega(k)}$ under ETH

3 Smaller Kernels:

- Current: $O(k^2)$ kernel
- Question: Linear kernel possible?

4 Directed FVS:

- Tournaments: Open complexity questions
- Better algorithms needed

Practical Research Directions

① Machine Learning Integration:

- Learn good branching rules
- Neural networks for FVS prediction
- Graph neural networks

② Dynamic FVS:

- Graph changes over time
- Incremental updates to FVS
- Online algorithms

③ Distributed Algorithms:

- Parallel FVS computation
- MapReduce framework
- Cloud computing

④ Application-Specific:

- Tailored algorithms for domains
- Exploit structure in real networks
- Integration with existing tools

Our Potential Contributions

After implementing and evaluating our algorithms, we plan to:

① **Comprehensive Benchmark:**

- Large-scale comparison on diverse graphs
- Publicly available dataset
- Reproducible results

② **Hybrid Algorithm:**

- Combine exact + metaheuristic
- Use exact for small subproblems
- Metaheuristic for large parts

③ **Real-World Case Studies:**

- Apply to local network data
- Demonstrate practical value
- Potential publication

④ **Open-Source Implementation:**

- Well-documented code
- Easy-to-use library

Summary

Problem

Feedback Vertex Set: Find minimum vertices to remove to make graph acyclic

Complexity

- NP-complete (general graphs)
- FPT: $O(3.619^k \cdot n^4)$
- 2-approximation polynomial algorithm

Algorithms

Surveyed 25+ algorithms across 5 categories:

- Exact, Approximation, Randomized, Heuristic, Metaheuristic
- Implementing: Iterative Compression + Genetic Algorithm

Applications

Project Timeline

Week	Milestone	Deliverables
6	Checkpoint	This presentation
7-8	Implementation	Working algorithms
9-10	Experimentation	Run all experiments
11	Analysis	Statistical analysis, plots
12	Documentation	Final report, code docs
13	Final Presentation	Complete results

Current Progress

- Problem understanding
- Literature review
- Algorithm survey
- Experimental design
- Next: Implementation begins

Thank you for your attention!

Questions and Discussion

Contact:

Your Email

Your Website/GitHub

Backup: Detailed Algorithm Comparison

Algorithm	Time	Space	Quality	Practical	Best For
Brute Force	$O(2^n n^3)$	$O(n^2)$	Optimal	$n \leq 15$	Tiny instances
DP Subsets	$O(2^n n^2)$	$O(2^n)$	Optimal	$n \leq 20$	Small instances
Iter. Compression	$O(5^k k n^2)$	$O(n^2)$	Optimal	$k \leq 30$	Small k
Bounded Search	$O(4^k n^2)$	$O(n^2)$	Optimal	$k \leq 35$	Small k
Current Best FPT	$O(3.619^k n^4)$	$O(n^2)$	Optimal	$k \leq 50$	Research
2-Approximation	$O(n^2)$	$O(n)$	$2 \times \text{OPT}$	Always	Speed + guarantee
LP Rounding	$O(n^{3.5})$	$O(n^2)$	$2 \times \text{OPT}$	$n \leq 10000$	Medium size
Greedy	$O(n^2 \log n)$	$O(n)$	No bound	Always	Very fast needed
Genetic Alg.	$O(gpn^2)$	$O(pn)$	Good	$n \leq 10000$	Large instances
Simulated Ann.	$O(in^2)$	$O(n)$	Good	$n \leq 10000$	Large instances

Backup: More Applications

① Compiler Optimization:

- Control flow graph analysis
- Loop optimization
- Used in GCC, LLVM

② Ecology:

- Keystone species identification
- Food web analysis
- Conservation priority

③ Transportation:

- Traffic light optimization
- Emergency vehicle routing
- Supply chain logistics

④ Quantum Computing:

- Circuit optimization
- Error propagation analysis

Backup: Parameter Settings

Genetic Algorithm Parameters

- Population size: 100
- Generations: 500
- Crossover rate: 0.8
- Mutation rate: 0.05
- Selection: Tournament (size 3)
- Elitism: Top 10%

Iterative Compression Parameters

- Maximum k : 50
- Branching strategy: Min-cut based
- Reduction rules: All standard rules
- Early termination: Yes (if k exceeded)